

Data Analytics

Data Integration (Fortsetzung)

Prof. Stefan Keller, FS26

*In Data Science, 80% of time spent
prepare data, 20% of time spent complain
about need for prepare data.
– Big Data Borat (@BigDataBorat)*

Inhalt Datenintegration (DI) (Fortsetzung)

- Tools für die Datenintegration aus VoWo 07 (Update)
- Tools für die Datenintegration Fortsetzung
- Datenaufbereitung und Datenreinigung
- Duplikaterkennung und Zeichenkettenvergleich
- Datenabgleich (Data Matching)
- Datenanreicherung (Data Enrichment)
- Synthetische Datengenerierung

(Fortsetzung der Tools-Vorstellung aus VoWo 07)

TOOLS FÜR DIE DATENINTEGRATION

- Data preparation (browsing)
 - Command-line Interface: Text editors, csvkit, DuckDB
 - GUI: MS Excel, OpenRefine, Open Data Editor (OKFN)
- Datenflussmanagement ("Workflow-Orchestrierung", Auswahl)
 - Apache Airflow: Flexible Orchestrierung komplexer Workflows, Monitoring, zahlreiche Integrationen
 - Apache NiFi: Einfache, webbasierte grafische Benutzeroberfläche, Monitoring, Echtzeitverarbeitung, skalierbar durch Clustering
- Dataprocessing – nur Transformation (Auswahl)
 - IBM InfoSphere DataStage: Starke Integration in IBM-Ökosystem, hohe Skalierbarkeit, Governance-Funktionen (Metadaten)
 - OpenRefine: Interaktive Datenbereinigung, umfangreiche Transformationstools, einfaches tabellarisches GUI, No-Code
 - dbt: Fokus auf Transformationen innerhalb Data Warehouses, gute Integration in Cloud und in CI/CD (oft mit Airflow)
 - Looker Studio (Google): Direkte Anbindung an Google Cloud-Dienste, intuitive Drag-and-Drop-Oberfläche, interaktive Dashboards
- Hybrid Dataprocessing und Datenflussmanagement (Auswahl)
 - KNIME: Starke Datenanalyse- und Visualisierungsfähigkeiten, umfangreiche Erweiterungen, kommerzieller Support
 - Microsoft SQL Server SSIS: Nahtlose Integration mit dem Microsoft-Ökosystem, starke Visualisierungs- und Debugging-Tools
 - Apache Hop: Grafische ETL-Entwicklung, kombiniert Datenflussmanagement mit Dataprocessing
 - FME by Safe Software: Unterstützung zahlreicher Datenformate, insbes. Geodatenformate, grafischer Workflow-Designer
 - Informatica PowerCenter: Robuste Datenmanagementfunktionen, starke Integration in bestehende Unternehmensarchitekturen

Tools für die Datenintegration: Fortsetzung

Toolname	Kategorie	Bemerkungen	Desktop / Webapp	Skalierbarkeit	OSS / komm.	Community	Sprache
Apache Airflow	Datenfluss-management	Workflow- und Prozess-Orchestrierung; weniger Transformation	Webapp	skalierbar, Batch und teilweise Echtzeit durch eigene Erweiterungen möglich	OSS	sehr aktiv	Python
Apache NiFi	Datenfluss-management	Datenfluss-Management mit umfangreichen Monitoring-Funktionen	Webapp	skalierbar, unterstützt sowohl Echtzeit- als auch Batch-Verarbeitung	OSS	sehr aktiv	Java
KNIME	Dataprocessing	Pipeline- und Prozess-management mit Integration analytischer Workflows	Desktop (OSS), Webapp (komm.)	Hauptsächlich Batch, Echtzeit mit zusätzlichem Aufwand möglich	OSS / komm.	aktiv	Java
Microsoft SQL Server SSIS	Dataprocessing	Starke Prozessverwaltung, jedoch limitiertes Datenfluss-Monitoring	Desktop (Windows), Visual Studio GUI	Primär Batch-Verarbeitung; Echtzeit durch zusätzliche Tools oder Services möglich	komm.	N/A	.NET
IBM InfoSphere DataStage	Dataprocessing	Umfassende Prozessverwaltung und Datenflussüberwachung	Desktop, Webapp	skalierbar; Echtzeit- und Batch-Verarbeitung möglich	komm.	N/A	Java
OpenRefine	Dataprocessing	Nur Transformationen und Bereinigung, keine Pipeline- oder Datenflussverwaltung	Desktop	Primär Batch, keine Echtzeitoptionen	OSS	Moderat aktiv	Java
dbt (Framework)	Transformation	Nur Transformation und Verwaltung von ELT-Pipelines, kein Datenflussmanagement	Batch-Tool; dbt Cloud Webapp	Hauptsächlich Batch; Cloud-native Ansätze möglich	OSS / komm.	sehr aktiv	Python
Looker Studio (Google)	Transformation / Visualisierung	Nur Transformation und Visualisierung, keine umfassende Pipeline- oder Datenflussverw.	Webapp (Cloud-only)	Cloud-basiert, Batch- und Echtzeit-Dashboards möglich	komm.	N/A	-
Apache Hop	Hybrid	Umfassende Pipeline-Definition und Datenfluss-Management	Desktop, Webapp, Batch-Tool	Batch-orientierte Verarbeitung; Echtzeit durch eigene Erweiterungen möglich	OSS	hohe Aktivität	Java
FME by Safe Software	Hybrid	Dataprocessing als auch gute Datenflusskontrolle	Desktop, Webapp	Gute Batch- und Echtzeit-Verarbeitung, für grosse Datenmengen optimiert	komm.	N/A	C++, Python
Informatica PowerCenter	Hybrid	Umfassende Datenprozesse und Datenflusskontrolle	Desktop, Webapp	skalierbar, sowohl Batch als auch Echtzeit	komm.	N/A	Java

Legende: **Orange** = wichtig hier; Community = Anzahl Github Points; komm. = kommerziell

Tools für die Datenintegration: Tipps zur Auswahl

- Zunächst wird die Kategorie betrachtet:
 - hier 1. Datenflussmanagement 2. Dataprocessing und 3. Hybrid.
 - Es gäbe auch weitere Funktionsgruppen, wie z.B. 4. Visualisierung.
- Dann gibt es Anforderungen/Kriterien wie v.a.
 - bestehendes Produkt-Ökosystem (Microsoft, IBM, Open Source, Google)
 - und in der Firma vorhandenes Know-how
 - Zur Erinnerung: kommerzieller Support gibt es auch für Open Source-Produkte.
- Schliesslich gibt es spezifische Anforderungen/Kriterien, wie z.B.
 - Geodaten, wo neben FME auch in Frage: Apache Hop, KNIME, Looker Studio.

DATENAUFBEREITUNG UND DATENREINIGUNG

- Datenintegration (Data Integration):
 - Verschiedene Daten, die von vielen Personen kontrolliert werden, zu einem Datensatz mit gemeinsamem Schema zusammenführen
 - Schwerpunkt hier, Datawarehouses: Teil Prof. S. Keller
- Datenaufbereitung (Data Wrangling) – Teil der Datenintegration:
 - Definition: Reinigung (Data Cleaning), Transformation und Anreicherung von Rohdaten vor der eigentlichen Verarbeitung und Analyse
 - Schwerpunkt hier, Datawarehouses: Teil Prof. S. Keller
- Datenreinigung (Data Cleaning) – Teil der Datenaufbereitung:
 - Fehlerhafte Daten erkennen und entfernen, damit ein zuverlässiger Datensatz entsteht, der keine fehlenden Werte enthält und alle Werte numerisch sind.
 - Schwerpunkt Data Mining / Machine Learning: Teil Prof. Politze

Datenfehler: Typen und Klassifikation

- Nachdem bisher die Überwindung der Heterogenität zwischen Schemata im Fokus lag, geht es hier um die eigentlichen Daten
- Jetzt stellt man häufig fest, dass weitere Probleme vorhanden sind:
 - Probleme mit der Qualität und der Vollständigkeit => Validierung
 - Datenfehler => Datenreinigung (Data Cleaning)
- Fehler sind in grossen Datenmengen allgegenwärtig
 - Einfache und schwere Fehler
 - Einfache Fehler: Durch Betrachtung eines Tupels erkennbar
 - Schwierige Fehler: Erst erkennbar durch Betrachtung mehrere Tupel
 - Systematische und unsystematische Fehler
 - Unsystematische sind oft schwierige Fehler
 - Systematische sind «einfach» zu korrigieren (nicht intuitiv 😊)

Datenaufbereitung: Leicht unterschiedliche Ziele

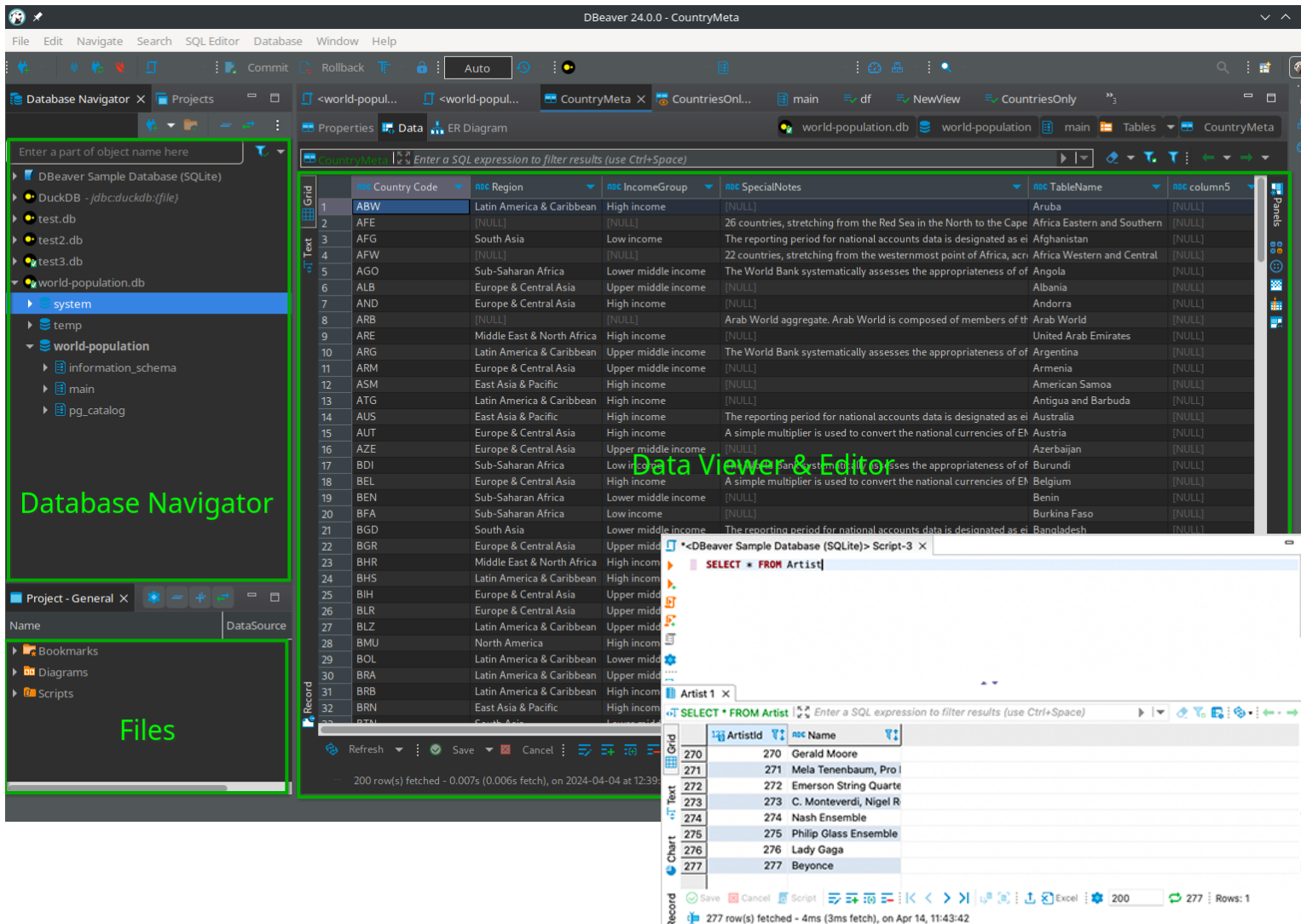
für Data Warehouses/Data Lakes (DW/DL) vs. Machine Learning (ML)

- Gemeinsamkeiten von DW/DL und ML:
 - erfordern die Integration/Konversion von Daten aus verschiedenen Quellen
 - erfordern oft die Transformation von Daten, um sie in ein einheitliches Format zu bringen
 - erfordern die Validierung und Bereinigung von Daten
- Unterschiede in der Datenaufbereitung für DW, DL und für ML :
 - Ziel für DW/DL ist die Bereitstellung von Daten **für Analysen**, während das Ziel für ML die Vorbereitung von Daten **für die Verwendung in maschinellen Lernmodellen** ist.
 - Datenaufbereitung für DW/DL erfolgt "**top-down**" (Zielschema vorgegeben), während sie für ML "**bottom-up**" erfolgt (Suche nach dem grössten gemeinsamen Nenner).
 - Für DW liegt der Schwerpunkt in Quell-/Zahlen-**Aggregation und Optimierung**; bei DL der Angabe **von Metadaten zu den Rohdaten**. Für ML liegt der Schwerpunkt oft auf der Dimensionalitäts-Reduktion; Umwandlung in numerische Merkmale; Data-Wrangling und Feature-Engineering.
 - Die Datenaufbereitung für DW/DL ist oft **ein einmaliger Prozess**, bei dem die Daten in regelmässigen Abständen aktualisiert werden. Die Datenaufbereitung für ML ist oft **iterativ** und erfordert kontinuierliche Anpassungen während des Trainingsprozesses.

Datenaufbereitung: Data Cleaning

- Data Cleaning (auch: Data Cleansing, Data Scrubbing)
- Teil der Datenaufbereitung
- Korrektur inkorrekt, inkonsistenter oder unvollständiger Daten
- Techniken:
 - Konvertierung (unterschiedliche Datenformate) und Normalisierung (Datum; Währung)
 - Domänenunabhängige Bereinigung: Schlüssel- und Attributvergleich (Deduplizierung)
 - Domänenspezifische Bereinigung: z.B. mit Thesauri
 - Regelbasierte Bereinigung:
 - Auf welche Daten ist die Regel ausgerichtet: Quelldaten oder Zieldaten?
 - Welche Art von Bereinigungsaktion wird durchgeführt: Prüfung, Filterung, Korrektur?

Datenbereinigung mit DBeaver und DuckDB



The screenshot shows the DBeaver 24.0.0 interface. On the left, the 'Database Navigator' pane displays a tree structure of databases, including 'DuckDB - jdbc:duckdb:file'. The main area shows a table with columns: 'Country Code', 'Region', 'Income Group', 'Special Notes', 'TableName', and 'column5'. The table contains 32 rows of data. Below the table, a 'Files' pane shows the project structure. In the bottom right, a 'SQL Editor' window displays a query: `SELECT * FROM Artist`. The status bar at the bottom indicates '277 row(s) fetched - 4ms (3ms fetch), on Apr 14, 11:43:42'.

Interessante Eigenschaften von DuckDB:

- In-process analytical DBMS. Wie SQLite – nur besser.
- Liest/schreibt Datenquellen: CSV, Parquet, PostgreSQL, MySQL, SQLite, Files, Google Cloud Storage, httpfs (HTTP, S3, GitHub)
- Kennt modernes SQL mit CTE und Window-Functions
- Spezieller Datentyp UNION: ermöglicht es einer Spalte, mehrere Datentypen zu enthalten

(Siehe auch "ETL using SQL and DuckDB and DBeaver - An Exercise" von S. Keller.

https://md.coredump.ch/s/PB_hp_CYNP)

DUPLIKATERKENNUNG UND ZEICHENKETTENVERGLEICH

Duplikaterkennung und Zeichenkettenvergleich mit SQL

```
with duplicates as (  
    select  
        id,  
        row_number() over(partition by ename, dname, gender order by id desc)  
        as rownum  
    from emp  
)  
delete from emp using duplicates -- PG-specific syntax  
    where emp.id = duplicates.id and duplicates.rownum > 1  
    returning *;
```

- Hier geht's um **exaktes Matching** und es wird hier **der erste Match** behalten und die anderen gelöscht
- Als Match ist in "partition by" definiert: **ename || dname || gender** - sinngemäss String-konkateniert
- Demo: <https://www.db-fiddle.com/f/gwE6cma7y6EP7oxDJQ39Vo/2>
- Quelle: <https://sqlfordevs.com/delete-duplicate-rows>

→ Was ist wenn ein **nicht-exakter Match** nötig wird (z.B. wegen Tippfehler)? Antwort →

Duplikaterkennung und Zeichenkettenvergleich (String Matching)

- Duplikate sind verschiedene Tupel in einer Relation, die dasselbe Realweltobjekt repräsentieren
- Ironischerweise hat Duplikaterkennung selber Synonyme: Dublettenerkennung. Englisch: record linkage, object identification, named entity resolution, reference reconciliation
- Challenge wenn ein String nicht exakt matched und Tippfehler enthält!
- Ähnlichkeitsmasse (Similarity measures):
 1. Edit- bzw. transpositionsbasierte Ähnlichkeitsmasse: Smith-Waterman, Jaro-Winkler
 2. Token-basierte Ähnlichkeitsmasse : Jaccard, TF/IDF
 - sowie hybride Ähnlichkeitsmasse (Generalized Jaccard, Soft TF/IDF, Monge-Elkan) und phonetische Ähnlichkeitsmasse (Soundex)

Effektiv haben wir zwei Challenges: **Genauigkeit (Accuracy) und Skalierbarkeit**

Ähnlichkeitsmass 1.: Edit-Distanz-basiert

- Auch bekannt als Levenstein-Distanz
- $d(x,y)$ berechnet die minimalen Kosten für die Umwandlung von x in y unter Verwendung einer Folge von Operatoren, die jeweils 1 kosten
 - Löschen eines Zeichens
 - Einfügen eines Zeichens
 - Ersetzen eines Zeichens durch ein anderes
- Modelliert häufige Bearbeitungsfehler
- Kleinerer Bearbeitungsabstand bedeutet höhere Ähnlichkeit

Tools: Siehe PostgreSQL Levenstein-Distanz und z.B. das "Python Record Linkage Toolkit" <https://recordlinkage.readthedocs.io/>

Ähnlichkeitsmass 1.: Smith-Waterman, Jaro

- Vorherige Masse berücksichtigen globale Übereinstimmungen; versuchen, alle Zeichen von x mit allen Zeichen von y abzugleichen
- Für manche Fälle nicht gut geeignet
 - z.B. "Prof. John R. Smith, Univ of Wisconsin" und "John R. Smith, Professor"
 - Die Ähnlichkeitsbewertung wäre hier recht niedrig
- Smith-Waterman:
 - Idee: Suche nach zwei Teilzeichenfolgen von x und y, die am ähnlichsten sind
 - z.B. "John R. Smith" im obigen Fall finden => lokale Ausrichtung
- Jaro und Jaro-Winkler:
 - Hauptsächlich für den Vergleich kurzer Zeichenfolgen, z.B. Vor- und Nachnamen

Ähnlichkeitsmass 2.: Mengen-based

- Zeichenketten als Mengen (Sets) von Token betrachten
- Verwendung mengenbezogener Eigenschaften zur Berechnung von Ähnlichkeitswerten
- Berücksichtigung von Wörtern, die durch Leerzeichen getrennt sind
 - evtl. die Wörter "stemmen" (je nach Anwendung)
 - Entfernen gängiger Stoppwörter (z.B. das, und, von)
 - z.B. bei "david smith" werden die Token "david" und "smith" erzeugt
- Berücksichtigung von n-Grammen, Teilstrings der Länge n (z.B. n=3)
 - z.B. "david smith" => Menge der 3-Gramme {##d, #da, dav, avi, ..., h##}
- Jaccard: Sehr verbreitet!
- TF/IDF: Mass aus Information Retrieval

DATENABGLEICH (DATA MATCHING)

- Suche nach strukturierten Datenelementen, die sich auf dieselbe reale Entität beziehen
 - Entitäten können durch Tupel, XML-Elemente oder RDF-Tripel dargestellt werden, nicht durch Strings wie beim String-Matching
 - z.B. (David Smith, 608-245-4367, Madison WI)
vs. (D. M. Smith, 245-4367, Madison)
- Kommt in vielen Integrationsszenarien vor
 - Zusammenführung mehrerer Datenbanken mit demselben Schema
 - Zusammenführen von Zeilen aus Quellen mit unterschiedlichen Schemata
 - Abgleich einer Benutzerabfrage mit einem Datenelement

Datenabgleich (Data Matching)

- Gegeben sind zwei Tabellen X und Y mit identischen Schemata
 - angenommen, jedes Tupel in X und Y beschreibt eine Entität (z.B. eine Person);
 - Ziel: Alle Übereinstimmungen zwischen X und Y finden
- Unterschied zum Zeichenkettenvergleich (String Matching):
 - Theoretisch könnte man jedes Tupel als String behandeln und dann String Matching anwenden; dies erschwert jedoch die Anwendung anspruchsvoller Techniken und domänen-spezifischer Kenntnisse
- Hier zur Information vier Techniken zum Datenabgleich:
 1. Rule-based matching
 2. Learning-based matching
 3. Matching by clustering
 4. Probabilistic approaches to matching (Bayes)

DATENANREICHERUNG (DATA ENRICHMENT)

Datenanreicherung (Data Enrichment)

- Datenanreicherung:
 - Die Zusammenführung von Daten Dritter aus einer externen Datenquelle mit bestehenden Daten durch Abgleich (durch gemeinsame Verknüpfungsattribute)
 - Beispiele von Verknüpfungsattributen: Ortsname, Geburtsdatum, Koordinaten(!)
- ... v.a. durch Open Data, denn die nimmt immer mehr zu! Oberbegriff von
 1. Open Government Data: Offene Behördendaten. Beispiel: PLZ-Liste der Post
 2. Open Public Data (volunteered, crowdsourced): Offene Daten von freiwilligen "Bürgern". Beispiel: Wikipedia, Wikidata, Flickr, OpenStreetMap
 3. Open Corporate Data: Offene Firmendaten. Beispiel: Google Earth Engine
- Tipps: <https://github.com/rnckp/awesome-ogd-switzerland/>

Datenanreicherung (Data Enrichment) Forts.

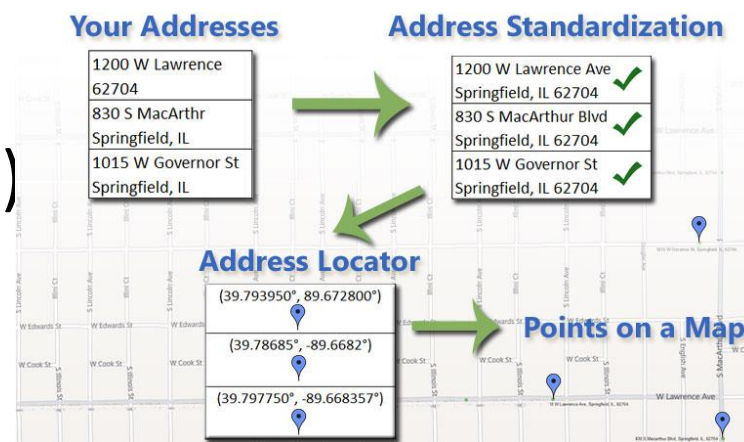
- Bereinigung – Beispiel
 - Gegeben **Kundenliste** mit Kundenname, ..., **PLZ+Ort**
sowie **Gebäudeadress-Daten** mit PLZ+Ort von Post und/oder OpenStreetMap
 - "Fuzzy Match"(!) von Kundenliste mit Gebäudeadress-Daten
 - Resultat: Kontrollierte Kundenliste
- Geocodierung (eigentliche Anreicherung) – Beispiel
 - Gegeben **Kundenliste** mit Kundenname, ..., **Strasse+PLZ+Ort**
sowie dieselben **Gebäudeadress-Daten** wie oben
 - Resultat: Angereicherte Kundenliste mit **Koordinate** (lat/lon)
 - Diese Koordinaten lassen sich auf Karte visualisieren (siehe nächste Folie)

Geocodierung

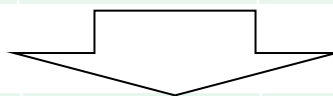
Der rechnerische Prozess der Transformation einer textuell-postalischen Adressbeschreibung in oder zu einen/m Ort auf der Erdoberfläche (Koordinaten)

Wichtige Komponenten:

- Adress-Standardisierer und Matcher-Methode
- Referenz-Datenbank mit Adr. (z.B. GWR, OSM)



CustID	Lastname	Firstname	Date_Birth	Nat	Gender	Kanton	Street	ZipCd	Place	Country	Phone		
101	Hauser	Pascal	20.05.1973	CH	M	ZH	Röschibachstr. 77	8037	Zürich	CH	044 530 81 97		
102	Ragginger	Jean-Pierre	29.09.1983	CH	M	ZH	Saatlenzelg 24	8050	Zürich	CH	044 290 10 52		
103	Faist	Jenny	08.10.1969	CH	F	ZH	Köschenrütistr. 69	8052	Zürich	CH	044 882 63 32		



CustID	Lastname	Firstname	Date_Birth	Nat	Gender	Kanton	Street	ZipCd	Place	Country	Phone	Latitude	Longitude
101	Hauser	Pascal	20.05.1973	CH	M	ZH	Röschibachstr. 77	8037	Zürich	CH	044 530 81 97	47.39337	8.52866
102	Ragginger	Jean-Pierre	29.09.1983	CH	M	ZH	Saatlenzelg 24	8050	Zürich	CH	044 290 10 52	47.412473	8.561729
103	Faist	Jenny	08.10.1969	CH	F	ZH	Köschenrütistr. 69	8052	Zürich	CH	044 882 63 32	47.42497	8.5368

Synthetische Datengenerierung

- Synthetische Daten versuchen, die allgemeinen Eigenschaften und Merkmale der Originaldaten zu erhalten, ohne Informationen über einzelne Datenproben preiszugeben
- Warum synthetische Daten? Echte Daten können Lücken aufweisen; der Zugang zu echten Daten kann eingeschränkt sein; echte Daten können dem Datenschutz unterliegen; echte Daten können nicht vorhanden sein
- Anwendungen: Modelltraining (ML/AI); Veröffentlichung offener Daten; Softwaretests; Datenbankoptimierung
- Methoden: Statistisch, regelbasiert, Datenerweiterung, Datenbank, maschinelles Lernen

Beispiel einer Datenbank-Methode "out-of-the-box" in PostgreSQL SQL:

```
SELECT
  round(random() * (upper_bound - lower_bound) + lower_bound)::int AS i,
  random() AS d,
  upper(substr(md5(random())::text),1,upper_bound)) AS str,
  random()>0.5 as bool,
  '2023-04-04'::timestamp + (random() * interval '1 month') - interval '1 months' as dt
FROM generate_series(1, 5) AS series,
LATERAL (select 1 as lower_bound, 4 as upper_bound) as bound
ORDER BY dt DESC;
```

i	d	str	bool	dt
2	0.11543211586548807	11DF	f	2023-03-30 12:48:59.5008
2	0.9191493654545013	0BAB	f	2023-03-24 20:05:44.7936
2	0.21507179287043243	0A25	f	2023-03-23 07:37:32.4768
4	0.15765081369994505	6E9B	t	2023-03-19 17:01:08.9184
3	0.7192091105639093	962A	t	2023-03-06 15:40:20.928

(5 Zeilen)

Synthetische Datengenerierung: Typen

- Typen von synthetisierten bzw. synthetischen Daten:

1. Anonymized data



513XXX	24XX	X	[30-40]	480.34	2300.56
--------	------	---	---------	--------	---------

2. Artificial data



41554523	4324	Rachel McDonald	100	1043	514
----------	------	-----------------	-----	------	-----

3. (Pure) Synthetic data



3253215	456	Jack Ma	33	558	2152
5453472	623	Mike Brown	34	603	27600

- 2. und 3. sind synthetische Daten, die nicht auf echten Daten beruhen

Synthetische Datengenerierung: Tools

- Python:
 - Faker lib. - Package that generates fake data, like "first names". MIT Lic., <https://faker.readthedocs.io/>
 - Synthetic Data Vault (SDV)– Tools to generate tabular synthetic data. Business source license, <https://docs.sdv.dev/sdv/>
- Webbased: Viele kommerzielle Angebote...
- PostgreSQL:
 - Plain SQL! Tipp `generate_series()`
 - PostgreSQL Anonymizer: PostgreSQL lic., Ruby (by Dalibo), https://labs.dalibo.com/postgresql_anonymizer
 - PGFaker: MIT Lizenz, TypeScript, <https://github.com/imanpalsingh/pg-faker>
 - pgsynthdata: MIT Lizenz, Python, by OST, <https://gitlab.com/geometalab/pgsynthdata>
Syntax: **COMMENT ON COLUMN** `my_table.my_column` IS ' ... ';

PostgreSQL Anonymizer

Gehört zum Type "1. Anonymized Data" (gemäss vorletzter Folie)

Syntax: **SECURITY LABEL FOR** anon -- my anon. schema
ON COLUMN customer.lastname
IS 'MASKED WITH FUNCTION anon.fake_last_name()';

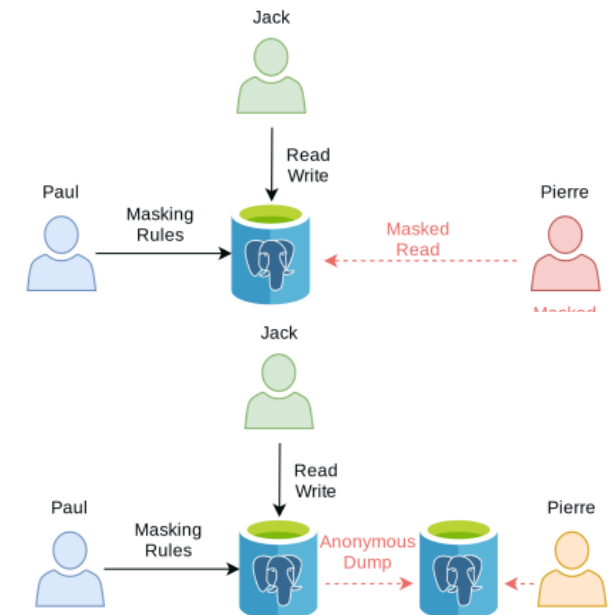
1. Statische Maskierung: Entferne die persönlich identifizierbare Informationen gemäss den Regeln für immer(!)
2. Dynamische Maskierung: Verstecke persönlich identifizierbare Informationen nur für die maskierten Benutzer
3. Generalisierung: Erstelle "verschwommene Ansichten" der Originaldaten; ähnlich zu Dynamische Maskierung; für Statistik und ML
4. Anonyme Dumps: Exportiere die maskierten Daten in eine SQL-Datei; für CI/CD (mein Favorit)

```
postgres=# SELECT * FROM customer;
 id | full_name | birth | employer | zipcode | fk_shop
----+-----+-----+-----+-----+-----
 911 | Chuck Norris | 1940-03-10 | Texas Rangers | 75001 | 12
 312 | David Hasselhoff | 1952-07-17 | Baywatch | 90001 | 423
(2 rows)

postgres=# SELECT anon.anonymize_database();
 anonymize_database
-----
 t
(1 row)

postgres=# SELECT * FROM customer;
 id | full_name | birth | employer | zipcode | fk_shop
----+-----+-----+-----+-----+-----
 911 | nicolau Lipphardt | 1940-03-10 | Isologics | 27695 | 12
 312 | daividh Meador | 1952-07-17 | Leaf and Stalk | 76437 | 423
(2 rows)

postgres=#
```



- Datenaufbereitung und Datenreinigung: Klassierungen (Datenfehler) und Abgrenzungen der DI kennenlernen
 - Duplikaterkennung und Zeichenkettenvergleich
 - Datenabgleich (Data Matching)
 - Datenanreicherung (Data Enrichment)
 - Synthetische Datengenerierung
-
- Dies ist der Abschluss vom Modul DatAna Teil 1 von mir (Prof. Keller)
 - Nächste Woche beginnt Teil 2 mit Prof. Politze